Scuola universitaria professionale della Svizzera italiana
Dipartimento tecnologie innovative

SUPSI

Branch-and-bound as PTAS

Studente/i

Sergio Aquilini

Tiziano Zamaroni

Relatore

Monaldo Mastrolilli

Correlatore

Koppányi István Encz

Eleonora Vercesi

Committente

SUPSI

Corso di laurea

C-P6051P - Seminari di Progetto

Modulo

M-P6050P - Progetto di Semestre

Anno

2025

Data

December 21, 2025

Contents

List of Figures

List of Tables

1 Introduction

Scheduling problems are a central topic in combinatorial optimization and arise in many real-world applications such as manufacturing systems, logistics, and computing infrastructures. Among them, parallel machine scheduling with the objective of minimizing the makespan is a fundamental NP-hard problem whose computational complexity grows rapidly with the size of the instance.

Branch-and-bound algorithms are a classical technique for solving NP-hard optimization problems. Traditionally, they are used as exact methods, providing optimal solutions at the cost of potentially exponential running time. As a consequence, their applicability is often considered limited to relatively small instances.

Recent research has proposed a different perspective, showing that branch-and-bound algorithms can be interpreted as approximation schemes when equipped with suitable bounding and stopping criteria. In particular, it has been shown that standard branch-and-bound frameworks can exhibit *Polynomial-Time Approximation Scheme* (PTAS) behavior, producing solutions arbitrarily close to the optimum within polynomial time for fixed approximation parameters [1].

This project builds upon that theoretical foundation and focuses on the practical behavior of branch-and-bound algorithms for parallel machine scheduling. While the reference work provides theoretical guarantees for the unrelated machines case, it also proposes additional optimizations for the uniform machines setting, aimed at improving search efficiency through the identification of structurally similar subproblems.

In this work, we focus on the identical machines case, which represents a special case of uniform machines. This choice allows us to experimentally validate the proposed optimizations in a simplified and controlled setting, isolating their effect from additional sources of complexity introduced by machine heterogeneity.

The objective of this project is to implement a branch-and-bound algorithm for identical machines scheduling, integrate the proposed profiling-based optimizations, and assess their impact through systematic computational experiments. The contribution of this work is thus

empirical, complementing the existing theoretical results with experimental evidence.

The remainder of this report is structured as follows. Chapter 2 presents the motivation and context of the project. Chapter 3 introduces the scheduling problem and the theoretical foundations of the algorithms considered. Chapters 4 and 5 describe the project requirements and the technologies used. Chapter 6 details the implementation, while Chapter 7 discusses the experimental results. Finally, Chapter 9 reflects on the outcomes of the project.

2 Motivation and context

2.1 Motivation

Branch-and-bound algorithms are traditionally regarded as exact methods for solving NP-hard combinatorial optimization problems. Due to their worst-case exponential complexity, their applicability is often considered limited to relatively small instances. Nevertheless, both empirical evidence and practical experience show that, when combined with appropriate bounding techniques and search strategies, branch-and-bound algorithms frequently perform significantly better than worst-case analysis would suggest.

A recent contribution by Encz, Mastrolilli, and Vercesi [1] introduces a novel and theoretically grounded interpretation of this phenomenon, showing that standard branch-and-bound frameworks can exhibit *Polynomial-Time Approximation Scheme* (PTAS) behavior for several combinatorial optimization problems, including parallel machine scheduling. In particular, their work demonstrates that branch-and-bound algorithms can be stopped early while still guaranteeing solutions arbitrarily close to the optimum, provided that suitable approximation criteria are employed.

While the original paper provides strong theoretical results supported by computational experiments, its primary focus lies on the unrelated and uniform parallel machine scheduling problems. The case of identical machines, although covered from a theoretical perspective, is not explored in the same depth from an empirical standpoint. Moreover, several optimizations suggested in the paper—such as profiling-based pruning and similarity-aware prioritization of search nodes—are not exhaustively validated through dedicated experimental studies for the identical machines setting.

The motivation of this semester project is therefore twofold. First, we aim to provide a *practical and empirical validation* of the theoretical claims presented in the reference paper. Second, we focus specifically on the uniform and identical parallel machine scheduling problems, implementing the additional optimizations proposed by the authors and systematically evaluating their impact on the behavior of the branch-and-bound algorithm.

This project does not aim to introduce new theoretical results. Instead, it emphasizes faithful

implementation, careful adaptation of existing techniques, and rigorous experimental analysis. The objective is to bridge the gap between theoretical guarantees and observed performance, and to assess whether the proposed enhancements effectively improve scalability and approximation quality in practice.

2.2 Context

This work was carried out as a semester project within the Bachelor of Science in Computer Science program at SUPSI.

The reference work for this project is *"Branch-and-Bound Algorithms as Polynomial-Time Approximation Scheme"* by Encz, Mastrolilli, and Vercesi [1]. That paper establishes that, for several classes of scheduling and knapsack problems, carefully designed branch-and-bound algorithms can be interpreted as approximation schemes with provable guarantees. In particular, it introduces profiling-based techniques that allow the algorithm to identify and avoid exploring structurally similar subproblems, thereby reducing redundant exploration of the search space.

Within this broader framework, the present project concentrates on the identical parallel machine scheduling problem, which can be seen as a special case of the uniform machines setting in which all machines have identical capabilities. This restriction simplifies certain aspects of the problem while preserving its NP-hard nature, making it a suitable case study for investigating the practical impact of approximation-oriented branch-and-bound techniques.

Concretely, the project involved the implementation of a branch-and-bound algorithm adapted to identical machines scheduling, the integration of profiling-based pruning and prioritization mechanisms proposed in the reference paper, the design of a flexible instance generation and experimental configuration pipeline, and the systematic collection and analysis of empirical data across multiple problem sizes and approximation parameters.

The result is an experimental study that complements the theoretical contributions of the reference work by providing empirical evidence on the effectiveness of profiling-enhanced branch-and-bound algorithms in the identical machines case, and by highlighting their practical behavior under realistic computational constraints.

3 Algorithms

In this chapter, we aim to describe the theoretical aspects of the project, particularly the algorithms used to solve the scheduling problem addressed and the underlying mathematical concepts.

Notation

Unless stated otherwise, we use the following conventions in the theoretical chapters.

- Machines are indexed by $i \in \{1, \dots, m\}$ and jobs by $j \in \{1, \dots, n\}$.
- For unrelated machines, the processing time of job j on machine i is $p_{j,i}$. For identical machines, we write p_j (equivalently $p_{j,i} = p_j$ for all i).
- Assignment variables are written as $x_{j,i}$. In relaxations, $x_{j,i} \in [0, 1]$ and $\sum_{i=1}^{m} x_{j,i} = 1$ for each job j.

3.1 Problem introduction

The problem considered in this project is a scheduling problem, namely a *parallel machines scheduling problem*. It involves a set of n jobs $\mathcal{J}$ that need to be assigned to a set of m machines $\mathcal{M}$. Each job has an associated processing time depending on which machine it is assigned to.

The objective of the scheduling problem is to minimize the makespan (the cost, i.e. the completion time), which is defined as the maximum completion time across all machines.

3.1.1 Instance definition

An *instance* of the scheduling problem can be informally described as numerical data that characterizes the specific scenario to be solved. In our case, an instance is defined by the processing times of each job on each machine, hence describing the costs of assigning jobs to machines:

- Matrix of the *processing times* of job j run on machine i, denoted as $p_{j,i}$ for $j = 1, 2, \ldots, n$ and $i = 1, 2, \ldots, m$.

3.1.2 Problem difficulty

A scheduling problem, and therefore its instances, can be easy to solve or hard to solve depending on the characteristics of the problem itself. Some characteristics that influence the difficulty are

- **Job processing times** - variation in processing times across jobs,
- **Machine capabilities** - difference between machines involved,
- **Instance size** - number of jobs and machines involved.

Next, we consider a few scenarios to illustrate how the above-mentioned characteristics affect a problem's difficulty.

3.1.2.1 Job processing times

Before we discuss the impact of job processing times on problem difficulty, we restrict the scope of this project to deterministic processing times, meaning that the processing time for each job on each machine is fixed and known in advance.

We distinguish between small and big jobs based on their processing times. We defer the exact definition of small and big jobs to a later section (see Section 6.2.5.2), as it relies on the approximation parameter ϵ in the context of profiling.

The ratio between small and big jobs leads to different scenarios:

- **EASY - Homogeneous jobs**: Small and big jobs have roughly the same processing times, leading to a more uniform distribution of workload across machines. This scenario simplifies the assignment process, as there are fewer variations to consider.
- **HARD - Mixed jobs**: A significant variation in processing times among jobs, with a mix of small and big jobs. This scenario introduces complexity in balancing the load across machines, as assigning big jobs to certain machines can lead to bottlenecks.
- **EASY - Dominant big jobs**: The presence of a few big jobs that dominate the processing times, while the rest are small. In this case, the focus shifts to efficiently scheduling the big jobs, while small jobs can be more flexibly assigned.

3.1.2.2 Machine capabilities

Machine capabilities play an important role in the complexity of the scheduling problem. As machines become less uniform and more specialized, the problem shifts from a simple sequencing task to a complex assignment problem. We identify three main scenarios, enumerated from easiest to hardest:

- **Identical machines**: All machines have the same capabilities, meaning a job's processing time is the same on any machine. This scenario simplifies the scheduling process, as any job can be assigned to any machine without affecting the overall makespan.
- **Uniform machines**: Machines have different but fixed capabilities (e.g. speeds; the speed is a machine characteristic and is the same across all jobs), leading to varying processing times for jobs depending on the machine they are assigned to. This scenario introduces complexity in balancing the load across machines, as some machines may become bottlenecks.
- **Unrelated machines**: Each machine has unique capabilities, meaning processing time of a job varies significantly depending on the machine it is assigned to. This scenario represents the most complex case, as it requires careful consideration of both job assignments and machine capabilities to minimize the makespan effectively.

For the sake of this project, we only focus on the first case of *identical machines*.

To summarize, the interplay between job processing times and machine capabilities significantly influences the complexity of the scheduling problem. By focusing on identical machines, we aim to simplify the analysis while still addressing a meaningful subset of the problem.

3.1.2.3 Instance size

Let's begin by discussing the *size of the problem*, which is arguably the easier influential factor to visualize. We approach this by distinguishing first the growth in size, and then the ratio between jobs and machines.

In the first case, it is intuitive that as the number of jobs and machines increases, the complexity of finding an optimal solution also increases. This is due to the exponential growth of possible assignments, with a search space that grows combinatorially with n and m.

In the second case, we rely on heuristic intuition rather than formal theoretical results. Empirical observations and practical experience suggest three typical scenarios:

- **EASY – Machines abundance** ($n/m \to 0$): A large number of machines is available to accommodate the jobs, leading to limited contention and simpler assignment strategies.
- **HARD – Bottleneck** ($n/m \approx 1 \dots 1.5$): Significant contention among machines leads to many competing assignments with similar quality, making it difficult to identify good schedules early in the search.
- **EASY – Jobs abundance** ($n/m \to \infty$): Many jobs compete for a limited number of machines, leading to machine saturation. In this regime, most machines are heavily loaded regardless of the specific assignment, and many schedules are near-optimal.

For the hardest regime, the ratio $n/m \approx 1 \dots 1.5$ is commonly regarded as particularly challenging from a heuristic and empirical perspective, although this is not a strict rule and depends on additional factors such as job processing times and machine capabilities.
If we consider the case of identical machines, which is the main focus of this project, the above heuristic needs to be adjusted. In fact, when $n/m = 1$, each machine receives exactly one job, yielding a trivial instance that is easy to solve[1]. Based on preliminary experiments and discussions during the project, we therefore suggest that, for identical machines, the hardest instances occur around a ratio of $n/m \approx 2$.

3.2 Solution approximation

Scheduling problems are known to be NP-hard, which implies that finding an exact solution in reasonable time for large instances is often impractical. Approximation provides a compromise between accuracy and computational efficiency, allowing solutions close to the optimum to be obtained in significantly reduced times.

We say that an algorithm is a ρ-approximation if it guarantees that the solution it produces is within a factor of ρ of the optimal solution. Formally, for a minimization problem such as the one considered in our project, if $C_{\mathcal{A}}$ is the cost of the solution produced by the algorithm $\mathcal{A}$ and C_{OPT} is the cost of the optimal solution, then the algorithm is a ρ-approximation if $C_{\mathcal{A}} \leq \rho \cdot C_{OPT}$ where $\rho \geq 1$.

For the sake of simplicity and consistency with the branch-and-bound PTAS framework introduced by Encz, Mastrolilli, and Vercesi [1], we introduce the variable $\epsilon > 0$ such that $\rho = 1 + \epsilon$. This allows us to express the approximation ratio in terms of how much worse the algorithm's solution can be compared to the optimal solution. Therefore, an algorithm is a $(1 + \epsilon)$-approximation if it guarantees that $C_{\mathcal{A}} \leq (1 + \epsilon) \cdot C_{OPT}$, namely that the solution produced is at most ϵ times worse than the optimal solution.

To transition to specific approximation schemes, we categorize algorithms based on their computational complexity, providing a means of comparison between different approaches.

3.2.1 Approximation schemes

Approximation schemes categorize algorithms based on their computational complexity, thereby providing a means of comparison between different approaches. The two classes of approximation schemes encountered in our project are *PTAS* and *FPTAS*. A brief description of each is provided below:

- **PTAS (Polynomial Time Approximation Scheme)**: An algorithm that, for every pos-

[1] In this case, the problem reduces to a one-to-one assignment with no scheduling complexity, and the makespan equals the longest job processing time.

itive ϵ value, produces a solution that is within a factor of $(1 + \epsilon)$ of the optimum in polynomial time relative to the input size. However, the execution time may depend exponentially (or worse) on $1/\epsilon$.

- **FPTAS (Fully Polynomial Time Approximation Scheme)**: An algorithm that, for every positive ϵ value, produces a solution within a factor of $(1 + \epsilon)$ of the optimum in polynomial time with respect to both the input size and $1/\epsilon$.

Hierarchically, FPTAS is a strict subset of PTAS (assuming $P \neq NP$), meaning that FPTAS offers a stronger guarantee on the algorithm's efficiency.

The difference between the two can be better perceived by looking at the case when a very accurate approximation is desired (i.e. ϵ very small, $\epsilon \to 0$): in this case,

- PTAS will take significantly longer to run, e.g. exponentially longer,
- FPTAS will take longer to run, but only polynomially longer.

3.3 Branch-and-bound

Branch-and-bound is the foundation of this project and is one of the numerous techniques available for solving combinatorial optimization problems, such as the scheduling problem. Other techniques include greedy algorithms, dynamic programming, and heuristic methods, which we will not mention in this context.

The branch-and-bound method is used to solve NP-hard problems. The algorithm systematically explores the solution space by dividing the problem into smaller subproblems (*branching*) and calculating lower and upper bounds to evaluate the quality of the solutions found (*bounding*). Once a subproblem's bound indicates that it cannot yield a better solution than the best one found so far, a new subproblem is selected for exploration. If a subproblem's bound is worse than the current best solution, it is ignored and the process continues (*selection/search*). This process continues until all subproblems have been explored or pruned, ultimately leading to the optimal solution.

From the brief description above, we can identify three main components of branch-and-bound:

- **Branching**: The division of the problem into smaller subproblems. In our context, this means to assign a specific job to a specific machine. We will refer to this as *fixing* a job to a machine.
- **Bounding**: The calculation of lower and upper bounds for each subproblem. These bounds help determine whether a subproblem can be discarded (pruning) or if it should be further explored.
- **Selection/Search**: The strategy used to select which subproblem to explore next. This can be done using different strategies, such as depth-first or breadth-first.

To better understand the algorithm's flow and its application to the scheduling problem, we delve into its specific characteristics in the next section.

3.3.1 Mathematical formulation

To precisely describe the branch-and-bound algorithm for the identical machines scheduling problem, we introduce the following mathematical notation:

- $x_{j,i} \in [0, 1]$ indicates the fraction of job j assigned to machine i
- $F \subseteq \{1, \ldots, n\} \times \{1, \ldots, m\}$ is the set of fixed assignments at a given node
- For a node with fixed assignments F, the residual problem considers only the unfixed jobs
- $LB(F)$ is the lower bound on the makespan for the subproblem with fixed assignments F
- $UB(F)$ is the upper bound on the makespan obtained by rounding a fractional solution
- GLB, GUB are the global lower and upper bounds on the optimal makespan

The algorithm maintains the following invariant: at any point during execution, the optimal makespan C^* satisfies:

$$\text{GLB} \leq C^* \leq \text{GUB} \tag{3.1}$$

Algorithm 1: Branch-and-bound for identical machines scheduling

Input: Instance with n jobs, processing times $p_1, \ldots, p_n$, and m identical machines

Output: Job-machine assignment minimizing makespan with quality guarantee $(1 + \varepsilon)$

▷ *Initialization*
Solve relaxed problem for root node $F = \emptyset$ to obtain lower bound $LB(\emptyset)$;
Round fractional solution to obtain feasible assignment with upper bound $UB(\emptyset)$;
Initialize queue $\mathcal{Q} \leftarrow \{(\emptyset, LB(\emptyset))\}$;
Set global bounds: GLB $\leftarrow LB(\emptyset)$, GUB $\leftarrow UB(\emptyset)$;
while $\mathcal{Q} \neq \emptyset$ *and* $\frac{GUB}{GLB} \geq 1 + \varepsilon$ **do**
  ▷ *Selection*
  Select node $(F, LB(F))$ from $\mathcal{Q}$ according to search strategy;
  ▷ *Branching*
  Choose fractional job $j^* \in \{1, \ldots, n\} \setminus \{j : (j, i) \in F \text{ for some } i\}$;
  ▷ *Bounding and Pruning*
  foreach *machine* $i \in \{1, \ldots, m\}$ **do**
    Construct child node $F' \leftarrow F \cup \{(j^*, i)\}$;
    if $|F'| = n$ **then**
      Compute makespan $C(F')$ of complete assignment F';
      if $C(F') <$ *GUB* **then**
        GUB $\leftarrow C(F')$;
    else
      Solve relaxed problem for F' to obtain $LB(F')$;
      if $LB(F') <$ *GUB* **then**
        Round to obtain feasible solution with $UB(F')$;
        GUB $\leftarrow \min\{\text{GUB}, UB(F')\}$;
        Add $(F', LB(F'))$ to $\mathcal{Q}$;
  ▷ *Update global lower bound*
  GLB $\leftarrow \min\{\text{GUB}, \min_{(F, LB(F)) \in \mathcal{Q}} LB(F)\}$;
return *Best assignment found with makespan GUB*;

3.3.2 Key mathematical components

3.3.2.1 Lower bound computation

For a node with fixed assignments F, the lower bound $LB(F)$ is computed by solving a relaxed version of the scheduling problem where jobs can be fractionally split across machines. Let $o_i = \sum_{(j,i) \in F} p_j$ denote the overhead (total processing time already fixed) on

machine i. The relaxed problem seeks to minimize:

$$\min_{x_{j,i} \geq 0} \max_{i=1,\ldots,m} \left\{ o_i + \sum_{j \notin \{j' : (j',i') \in F\}} p_j \cdot x_{j,i} \right\} \tag{3.2}$$

subject to:

$$\sum_{i=1}^{m} x_{j,i} = 1 \quad \forall j \notin \{j' : (j', i') \in F\} \tag{3.3}$$

This can be efficiently solved using greedy allocation or binary search techniques.

3.3.2.2 Upper bound via rounding

Given a fractional solution $\{x_{j,i}\}$ from the relaxed problem, an upper bound is obtained by converting it to a complete assignment. Since at most m jobs can be fractionally assigned, these are rounded to integral assignments using a matching-based approach that minimizes the resulting makespan.

3.3.2.3 Pruning criterion

A node $(F, LB(F))$ is pruned if:

$$LB(F) \geq \text{GUB} \tag{3.4}$$

This ensures that the subtree rooted at this node cannot contain a better solution than the current best.

3.3.2.4 Termination condition

The algorithm terminates when the approximation guarantee is satisfied:

$$\frac{\text{GUB}}{\text{GLB}} < 1 + \varepsilon \tag{3.5}$$

At termination, the solution with makespan GUB is guaranteed to be within a factor of $(1+\varepsilon)$ of the optimal makespan C^*.

4 Requirements

The semester project is carried out in the context of the paper *Branch-and-Bound Algorithms as Polynomial-Time Approximation Scheme* [1], written by our supervisor Prof. Monaldo Mastrolilli together with István Encz Koppányi and Eleonora Vercesi. The authors provide a Python implementation of a branch-and-bound algorithm for the *unrelated machines* case, including heuristics and bound management. Our task consists in thoroughly understanding this implementation, adapting it to the *identical machines* case, and integrating the *profiling* technique, which is applicable since identical machines represent a special case of uniform machines. The final objective is to experimentally validate the results presented in the paper by comparing the base version of the algorithm with the profiling-enhanced variant.

4.1 Functional requirements

The project involves the following functional requirements:

R1 – Solver for identical machines

Develop a variant of the branch-and-bound algorithm capable of solving the scheduling problem on identical machines, by adapting the structure and functions of the existing implementation for unrelated machines.

R2 – Profiling integration

Implement the profiling mechanism described in the paper, in order to obtain a second variant of the algorithm that is directly comparable with the base version.

R3 – Experiment generation and execution

Run both variants of the algorithm and record the results, collecting a sufficiently large and meaningful dataset.

R4 – Data collection

Store structured information such as the number of explored nodes, execution time, deviation from the optimal solution of the considered instance, and other relevant metrics.

R5 – Experimental validation

Analyze the collected data using statistical and visualization tools to assess the effectiveness of profiling compared to the base version of the algorithm.

4.2 Non-functional requirements

No formal non-functional requirements were defined; however, the project implied several practical constraints:

NFR1 – Computational efficiency

The implementation and the choice of instances had to allow the generation and analysis of a significant amount of data within the time constraints of the semester project.

NFR2 – Reproducibility

Instance generation and algorithm execution had to be reproducible through controlled random seeds.

NFR3 – Code maintainability and clarity

Although the project was not intended for future development, the code had to be written clearly and integrate coherently with the existing structure.

5 Technologies used

5.1 Algorithm implementation

The implementation of the algorithms was developed in *Python*, starting from the code provided by the authors of the paper [1], which was originally designed for the *unrelated machines* case. In that version, the lower bound of the branch-and-bound nodes was computed using the *SCIP* solver [2], accessed through the *PySCIPOpt* library. This choice, which is appropriate for the general problem, allows the authors to obtain fractional bounds by means of *binary search* and *linear relaxation*.

In our project, however, this approach proved to be overly expensive and not strictly necessary. In the case of *identical machines*, the structure of the problem is significantly simpler and allows the use of much lighter methods for lower bound computation. Moreover, the use of *SCIP* [2] introduced an additional practical limitation: it was not possible to force the solver to systematically use the *primal simplex* method, which is required to guarantee that the number of fractional variables in the LP solution remains less than or equal to the number of machines. The absence of this guarantee compromised the reliability of the rounding phase and introduced a non-negligible computational overhead.

For these reasons, we completely replaced the use of *SCIP* [2] with a *greedy*-based lower bound computation method, which is fully sufficient for the identical machines case and significantly more efficient from a computational perspective. This choice allowed us to drastically reduce the processing time of the branch-and-bound nodes while maintaining a bound quality adequate for the purposes of the project.

5.2 Data analysis

The analysis of the experimental data was carried out in Python using standard data science libraries. *numpy* was used for basic numerical processing, *pandas* for organizing and aggregating the results collected during the experiments, and *matplotlib* for generating the visualizations used to compare the base version of the algorithm with the profiling-enhanced

variant. The data were exported in CSV format and analyzed through dedicated notebooks, making it possible to summarize performance trends and quantitatively support the discussion of the results.

6 Implementation

Conventions

In the implementation, arrays are 0-indexed (machines $i \in \{0, \ldots, m-1\}$, jobs $j \in \{0, \ldots, n-1\}$), whereas the mathematical notation in this report is 1-indexed. To avoid ambiguity, we use:

- In the reference Python implementation, the global bounds are stored in variables named `LLB` and `LUB`. Throughout this chapter, we keep the same notation to stay close to the implementation; elsewhere in the report we refer to these quantities as GLB and GUB to emphasize their global scope:

$$\texttt{LLB} \equiv \mathrm{GLB}, \qquad \texttt{LUB} \equiv \mathrm{GUB}.$$

- Node-level bounds are denoted by LB and UB (or $\mathrm{LB}(N)$ and $\mathrm{UB}(N)$ when the node needs to be explicit), corresponding to the fields `Node.LB` and `Node.UB`.
- `fixed` denotes the set of fixed assignments (corresponding to F in the theoretical chapter).

6.1 Branch-and-bound

The theoretical foundations and correctness of the branch-and-bound algorithm are discussed in 3. In this section, we focus exclusively on its concrete implementation, data structures, and control flow.

The implementation employed in this project is based on a branch-and-bound framework originally developed for the *unrelated machines scheduling* problem. The same structure is later reused for the identical machines case, where only the lower-bound routines and selected optimization components (such as profiling) are adapted, while the core branching, bounding, rounding, and node-selection logic is preserved.

6.1.1 Implementation overview

Each node of the branch-and-bound tree represents a subproblem corresponding to a partial assignment of jobs to machines and is characterised by:

- a set of fixed assignments `fixed` $\subseteq \{0, \dots, n-1\} \times \{0, \dots, m-1\}$,
- an overhead vector $\mathbf{o} \in \mathbb{R}^m$, where o_i denotes the load already assigned to machine i,
- a fractional solution X^{frac} obtained from a relaxation of the problem,
- a node lower bound LB and a node upper bound UB,
- the depth of the node in the search tree.

For a given node, the lower bound LB and upper bound UB correspond to the quantities defined in Chapter 3, and are computed here according to the selected bounding and rounding strategies.

The global state maintains the incumbent solution and the associated global bounds LUB and LLB, as defined in Chapter 3. In the implementation, LLB is derived from the set of active nodes and clamped by LUB for numerical robustness.

$$\mathrm{LLB} \leftarrow \min\bigl(\mathrm{LUB}, \min_{N \in Q} \mathrm{LB}(N)\bigr),$$

The behavior of the algorithm is governed by four main components. The following components correspond to the theoretical elements introduced in Chapter 3, and are described here only in terms of their implementation.

Bounding. For each node, a lower bound on the optimal makespan of the associated subproblem is computed either via a linear relaxation or via a feasibility-based binary search (see Section 6.1.3). Nodes whose lower bound is not strictly smaller than the current global upper bound LUB are pruned.

Branching. When the relaxation at a node yields a non-integer solution, one fractional job is selected according to a branching rule (see Section 6.1.5). This job is fixed on each possible machine, generating up to m child nodes and partitioning the remaining search space.

Rounding. A feasible integer solution is obtained from the fractional relaxation through a deterministic rounding strategy (see Section 6.1.4). The resulting makespan defines a node upper bound UB, which may improve the current global best solution.

Node selection. Active nodes are stored in a priority queue and extracted according to a configurable selection strategy (see Section 6.1.6). The choice of strategy affects the traversal order of the search tree but not the correctness of the algorithm.

The algorithm terminates according to the relative optimality gap condition introduced in Chapter 3:

$$\frac{\text{LUB}}{\text{LLB}} < 1 + \varepsilon.$$

In practice, an additional limit on the maximum number of explored nodes may be enforced to prevent excessive runtimes.

6.1.2 Pseudocode

Algorithm 2: Branch-and-bound framework (unrelated machines scheduling)

Input: instance $p_{j,i}$, tolerance ε; strategies for node selection, bounding, branching, rounding

Output: best solution $X^\star$ and its objective value LUB

$(X^{\text{frac}}, \mathrm{LB}) \leftarrow \texttt{LowerBound}(p, o{=}0, \texttt{fixed}{=}\emptyset)$;
if X^{frac} *is integral* **then**
  return $(X^{\text{frac}}, \mathrm{LB})$;
$(X^\star, \mathrm{LUB}) \leftarrow \texttt{Rounding}(X^{\text{frac}}, \emptyset)$;
Initialize queue Q with root node $(\texttt{fixed}{=}\emptyset,\ o{=}0,\ X^{\text{frac}},\ \mathrm{LB})$;
$\mathrm{LLB} \leftarrow \mathrm{LB}$;
while $Q \neq \emptyset$ **do**
  $\mathrm{LLB} \leftarrow \min(\mathrm{LUB},\ \min_{N \in Q} N.\mathrm{LB})$; `// clamp by LUB for robustness`
  if $\mathrm{LUB}/\mathrm{LLB} < 1 + \varepsilon$ **then**
    break;
  Extract node N from Q according to the node selection strategy;
  Select a fractional job j from $N.X^{\text{frac}}$ (branching rule);
  for $q \in \{0, \ldots, m-1\}$ **do**
    $\texttt{fixed}' \leftarrow N.\texttt{fixed} \cup \{(j,q)\}$; `// create child by fixing` (j,q)
    $o' \leftarrow N.o$; $o'_q \leftarrow o_q + p_{j,q}$;
    if $fixed'$ *assigns all jobs* **then**
      $\mathrm{UB}' \leftarrow \max_i o'_i$;
      if $\mathrm{UB}' < \mathrm{LUB}$ **then**
        $(X^\star, \mathrm{LUB}) \leftarrow (\texttt{fixed}', \mathrm{UB}')$;
      continue;
    $(X^{\text{frac}\prime}, \mathrm{LB}') \leftarrow \texttt{LowerBound}(p, o', \texttt{fixed}')$;
    if $\mathrm{LB}' \geq \mathrm{LUB}$ **then**
      continue ; `// prune by bound`
    if $X^{frac\prime}$ *is integral* **then**
      if $\mathrm{LB}' < \mathrm{LUB}$ **then**
        $(X^\star, \mathrm{LUB}) \leftarrow (X^{\text{frac}\prime}, \mathrm{LB}')$;
      continue;
    $(X^{\text{int}\prime}, \mathrm{UB}') \leftarrow \texttt{Rounding}(X^{\text{frac}\prime}, \texttt{fixed}')$;
    if $\mathrm{UB}' < \mathrm{LUB}$ **then**
      $(X^\star, \mathrm{LUB}) \leftarrow (X^{\text{int}\prime}, \mathrm{UB}')$;
    Insert child node $(\texttt{fixed}', o', X^{\text{frac}\prime}, \mathrm{LB}')$ into Q;
return $(X^\star, \mathrm{LUB})$;

Algorithm 2 summarises the inherited branch-and-bound framework used for the unrelated machines scheduling problem. For each node, a relaxation is solved to obtain a fractional solution X^{frac} and a lower bound LB on the optimal makespan of the corresponding subproblem.

A global upper bound LUB is maintained through feasible integer solutions produced either by rounding fractional solutions (Section 6.1.4) or by integrality of the relaxation. Whenever a solution with smaller makespan is found, the incumbent solution $X^\star$ and the global upper bound are updated.

The search space is explored by repeatedly extracting nodes from a priority queue according to the selected node selection strategy. If the extracted node is fractional, a branching rule selects one fractional job j; branching fixes j on each machine, generating up to m child nodes. For each child node, a new lower bound LB′ is computed and the node is immediately discarded if LB′ $\geq$ LUB, since it cannot lead to an improvement over the incumbent solution.

The global lower bound LLB is derived from the minimum lower bound among the nodes currently stored in the queue, clipped by LUB.

6.1.3 Lower bound strategies

The inherited implementation provides two alternative strategies to compute node-level lower bounds within the branch-and-bound framework. Both operate on the subproblem defined by:

- the set of fixed assignments `fixed`,
- the overhead vector $\mathbf{o} \in \mathbb{R}^m$,
- the remaining unfixed jobs.

Linear relaxation (`lin_relax`). In the linear-relaxation strategy, a linear program is solved in which each unfixed job may be fractionally assigned to machines. Let $x_{j,i} \geq 0$ denote the fraction of job j assigned to machine i, and let $C_{\max}$ denote the makespan. The model can be expressed as:

$$
\begin{aligned}
&\min C_{\max} \\
&\text{s.t.} \sum_{i=0}^{m-1} x_{j,i} = 1 && \forall j \text{ unfixed}, \\
&\quad \sum_{j \text{ unfixed}} p_{j,i} x_{j,i} \leq C_{\max} - o_i && \forall i = 0, \ldots, m-1, \\
&\quad x_{j,i} \geq 0 && \forall j, i.
\end{aligned}
$$

The fixed jobs do not appear as decision variables; their contribution is encoded in the overhead o. The optimal value $C^{\star}_{\max}$ is used as the node lower bound LB, and the corresponding solution defines X^{frac}. Due to the structure of the formulation, at most m jobs are fractional in the optimal solution.

Binary search on the makespan (`bin_search`). The second strategy performs a binary search on the makespan value. An interval $(\ell, r]$ is maintained such that ℓ is infeasible and r is feasible. For a candidate makespan C, a feasibility LP is solved with variables $x_{j,i}$ for unfixed jobs, subject to:

$$\begin{aligned}
&\sum_{i=0}^{m-1} x_{j,i} = 1 && \forall j \text{ unfixed},\\
&\sum_{j \text{ unfixed}} p_{j,i} x_{j,i} \le C - o_i && \forall i = 0, \dots, m-1,\\
&x_{j,i} = 0 && \text{if } p_{j,i} > C - o_i,\\
&x_{j,i} \ge 0 && \text{otherwise.}
\end{aligned}$$

If the LP is feasible, the value C becomes the new right endpoint; otherwise it becomes the new left endpoint. The process continues until $(\ell, r]$ cannot be further refined, and the final feasible value r is taken as the node lower bound. The corresponding LP solution defines the fractional assignment X^{frac}.

6.1.4 Rounding strategies

When the relaxation at a node yields a fractional solution X^{frac}, a deterministic rounding procedure is applied to construct a feasible integer assignment. The resulting schedule defines a node upper bound UB, which may improve the current global upper bound LUB.

The rounding implementation operates on the subproblem defined by the fixed assignments `fixed` and the overhead vector $o \in \mathbb{R}^m$. Jobs already fixed do not appear as decision variables; their contribution is accounted for by the overhead values o_i. Moreover, jobs that are already integrally assigned in X^{frac} are preserved.

Let F denote the set of *fractional jobs*, i.e., those jobs j such that $x_{j,i}$ is non-integer for at least one machine i. The implementation assumes $|F| \le m$, which is consistent with the structure of the relaxations used in Section 6.1.3. Rounding proceeds by first computing the machine completion times induced by (i) integrally assigned jobs in X^{frac} and (ii) fixed jobs, and then assigning the jobs in F according to one of the following strategies.

Assign-to-shortest machine (`all_to_shortest`). Each fractional job $j \in F$ is assigned to the machine that minimises its processing time:

$$i^\star(j) \in \arg\min_{i=0,\dots,m-1} p_{j,i}.$$

The completion times are updated accordingly, and the upper bound is obtained as the makespan of the resulting schedule:

$$\mathsf{UB} = \max_{i=0,\dots,m-1} C_i,$$

where C_i denotes the completion time of machine i. This strategy is computationally inexpensive and purely greedy with respect to individual job processing times.

Best matching of fractional jobs (`best_matching`). Since $|F| \leq m$, the fractional jobs can be assigned by enumerating all injective mappings of F to machines. Equivalently, all permutations of machines of length $|F|$ are considered. For each candidate placement, the completion times are updated and the makespan is evaluated; the assignment yielding the minimum makespan among the enumerated placements is selected, and the corresponding makespan is returned as UB.

6.1.5 Branching rules

When the relaxation at a node produces a non-integer solution, a set of *fractional jobs* is obtained, namely those jobs j for which at least one variable $x_{j,i}$ is non-integer. The implementation assumes that the number of such jobs is at most the number of machines m, which is guaranteed by the structure of the relaxations. A branching rule selects one job j from this set, and the node is branched by fixing job j on each machine $q = 0, \dots, m-1$, generating up to m children.

Two selection criteria are provided.

Maximum minimum processing time (`max_min_proc`). For each fractional job j, the quantity

$$\min_{i=0,\dots,m-1} p_{j,i}$$

is considered. The branching job is the one that maximises this minimum processing time. This rule gives priority to jobs that are relatively expensive even on their best machine, so that branching decisions focus on jobs with high potential impact on the makespan.

Maximum average processing time (`max_avg_proc`). For each fractional job j, the average processing time

$$\frac{1}{m}\sum_{i=0}^{m-1} p_{j,i}$$

is computed. The branching job is the one with maximum average processing time, i.e., the job that is globally heaviest across all machines. This tends to branch first on jobs that dominate the total processing workload.

6.1.6 Node selection strategies

The branch-and-bound framework explores the search space by maintaining a set of active nodes in a priority queue. The order in which nodes are extracted from this queue defines the exploration strategy. While node selection does not affect the correctness of the algorithm, it can significantly influence the order in which feasible solutions are found and, consequently, the effectiveness of pruning.

Each node is associated with a priority value derived from its attributes, and nodes are compared according to a configurable selection strategy. The implementation supports the following strategies.

Lowest lower bound first. Nodes are ordered by increasing node lower bound LB. At each iteration, the node with the smallest lower bound is selected. This strategy prioritizes subproblems that appear most promising with respect to the objective and is commonly used in best-first branch-and-bound variants.

Depth-first search. Nodes with larger depth in the search tree are prioritized. This strategy explores complete assignments early, which may quickly produce good feasible solutions and improve the global upper bound, potentially increasing pruning opportunities.

Breadth-first search. Nodes are selected in order of increasing depth. This strategy explores the search tree level by level and provides a more uniform coverage of the solution space, at the cost of potentially delaying the discovery of good feasible solutions.

6.2 Adaptation to identical machines scheduling

This section describes the adaptation of the inherited branch-and-bound framework to the *identical machines scheduling* problem, which constitutes the main scope of this project. While the overall structure of the algorithm remains unchanged, the identical-machines setting enables problem-specific simplifications and optimizations that significantly affect the

bounding procedures, rounding strategies, stopping criterion, and the integration of profiling mechanisms inspired by the PTAS described in the reference paper [1].

The contribution of this project lies in the faithful and robust translation of these ideas into an efficient implementation, addressing practical issues that arise.

6.2.1 Key differences with respect to the unrelated case

In the identical-machines setting, processing times depend only on the job index, i.e. $p_{j,i} = p_j$ for all machines i. This structural simplification leads to the following consequences:

- The fractional optimum for the root relaxation admits a closed form, given by the average load.
- Lower bound computations can be simplified and made solver-independent.
- Rounding strategies can exploit the symmetry between machines and rely on current machine loads.
- Profiling-based prioritization and pruning can be integrated to reduce the effective search space.
- The stopping criterion must be relaxed to account for the additional approximation introduced by profiling.

The following subsections describe these adaptations in detail.

6.2.2 Bounding for identical machines

6.2.2.1 LP-based approach and rounding incompatibility

An initial attempt to reuse the LP-based bounding strategies employed for the unrelated-machines case relied on solving linear relaxations via a generic LP solver. However, the rounding procedures used throughout the framework rely on the structural property that the number of fractional jobs in an optimal relaxation solution is bounded by the number of machines m.

In practice, the LP solver occasionally returned fractional solutions that violated this assumption, for instance by distributing a large number of jobs fractionally across machines. While such solutions are theoretically valid, they are incompatible with the in-place rounding procedures assumed by the framework, causing rounding to become ill-defined or inefficient.

Since enforcing a specific extreme-point structure on the solver output proved impractical, a different approach was adopted for the identical-machines case.

6.2.2.2 Greedy fractional bound

For identical machines, the relaxation admits a much simpler structure. Let

$$C_{\max}^{\text{frac}} = \frac{1}{m} \sum_{j=1}^{n} p_j$$

denote the fractional optimum makespan in the absence of fixed jobs. The greedy bound constructs a feasible fractional assignment that respects the current fixed assignments and machine loads, and fills the remaining capacity up to $C_{\max}^{\text{frac}}$.

The procedure operates as follows:

- The current machine loads induced by fixed jobs are stored in an overhead vector o.
- Unfixed jobs are processed sequentially.
- Each job is fractionally assigned to machines whose current load is below $C_{\max}^{\text{frac}}$, until its processing time is fully distributed.

The resulting fractional assignment is feasible by construction and yields a valid lower bound for the corresponding subproblem. Moreover, the greedy procedure is deterministic and produces fractional solutions compatible with the rounding strategies used in the framework.

Algorithm 3: Greedy fractional bound for identical machines

Input: processing times p_j, overhead o, number of machines m

Output: fractional assignment and its makespan

$C_{\max} \leftarrow \frac{1}{m} \sum_j p_j$;
$C_i \leftarrow o_i \quad \forall i \in \{0, \ldots, m-1\}$;
for *each unfixed job* j **do**
    $r \leftarrow p_j$;
    while $r > 0$ **do**
        Select a machine i with $C_i < C_{\max}$;
        $\delta \leftarrow \min(r,\ C_{\max} - C_i)$;
        Assign fraction δ / p_j of job j to machine i;
        $C_i \leftarrow C_i + \delta$;
        $r \leftarrow r - \delta$;
return fractional assignment and makespan $\max_i C_i$;

6.2.3 Rounding strategies for identical machines

Given a fractional solution X^{frac}, a deterministic rounding procedure is applied to obtain a feasible integer schedule. Jobs that are already integrally assigned in X^{frac}, as well as fixed jobs, are preserved. Let F denote the set of fractional jobs.

Assign-to-shortest machine (`all_to_shortest`). Each fractional job $j \in F$ is assigned to the machine with the smallest current completion time. This greedy strategy exploits the symmetry of identical machines and incrementally balances the load. The resulting makespan defines the node upper bound UB. This strategy is simple and efficient, providing a reasonable approximation in practice. It is the default rounding method we used throughout the experiments and in our analysis.

First-machine assignment (`first_machine`). Each fractional job is assigned to the first machine it appears on in the fractional solution. This rule provides a simple, deterministic baseline and is primarily used for testing and comparison.

6.2.4 Stopping criterion and approximation guarantee

In the absence of profiling, the stopping criterion coincides with the standard relative gap condition:

$$\frac{\mathsf{LUB}}{\mathsf{LLB}} < 1 + \varepsilon,$$

which guarantees a $(1+\varepsilon)$-approximate solution.

When profiling is enabled, additional pruning is performed based on profile equivalence. As prescribed by the reference paper, this introduces an extra approximation factor, which must be accounted for in the termination condition. Accordingly, the algorithm is allowed to terminate when

$$\frac{\mathsf{LUB}}{\mathsf{LLB}} < (1 + \varepsilon)^2.$$

For the same input value of ε, this condition admits a larger relative optimality gap than in the non-profiling case. The relaxed stopping criterion compensates for the approximation introduced by profile-based pruning and ensures that the final solution satisfies the approximation bound established in the paper.

6.2.5 Profiling-based prioritization and pruning

Profiling is introduced to reduce the effective search space by identifying nodes that induce similar load distributions. Each node is associated with a *profile key* that abstracts the distribution of machine loads induced by large jobs.

6.2.5.1 Profiling modes

Three profiling modes are supported:

- `NO_PROFILING`: profiling is disabled.
- `PRIORITY`: nodes without a previously seen profile are prioritized in the queue.

- `PRUNE`: nodes whose profile matches one already seen at the same depth are discarded.

6.2.5.2 Normalization and big jobs

Let

$$T_{\text{norm}} = \frac{1}{m} \sum_j p_j$$

denote the normalization scale. A job j is classified as *big* if its processing time is significant with respect to the normalization scale, namely if

$$p_j \geq \varepsilon \cdot T_{\text{norm}}.$$

This threshold follows the PTAS framework for identical machines, where jobs smaller than $\varepsilon \cdot T_{\text{norm}}$ are treated as negligible at the chosen approximation scale. Accordingly, only the contribution of big jobs is considered when computing profiling information.

For implementation efficiency, each node explicitly stores, in addition to the total machine loads o, a separate vector $o^{\text{big}} \in \mathbb{R}^m$ containing only the load contribution of big jobs. This vector is incrementally updated during branching and is used exclusively for profiling, so that profile computation does not require recomputing or filtering job assignments.

6.2.5.3 Geometric binning

Machine loads are normalized by T_{norm} and discretized using a geometric grid:

$$0, \varepsilon, \varepsilon(1+\varepsilon), \varepsilon(1+\varepsilon)^2, \ldots, 2(1+\varepsilon)^2.$$

This discretization preserves relative load differences while tolerating small perturbations, as required by the PTAS framework.

6.2.5.4 Construction of profile keys

For each node, profiling is based on the distribution of machine loads induced by big jobs only. Let $o^{\text{big}} \in \mathbb{R}^m$ denote the vector of machine loads obtained by summing only the processing times of big jobs assigned to each machine.

Each component is normalized as

$$\tilde{o}_i = \frac{o_i^{\text{big}}}{T_{\text{norm}}}.$$

The normalized loads are then mapped to the geometric bins, and a histogram over all machines is constructed. This histogram constitutes the *profile key* of the node.

6.2.5.5 Profile comparison and depth-based equivalence

Two nodes are considered ε-equivalent if they:

- occur at the same depth in the branch-and-bound tree, and
- share the same profile key.

The implementation maintains, for each depth, a set of profile keys that have already been observed. When a new node is generated, its profile key is computed and compared against this set. If an identical key is found, the node is marked as having a similar profile.

Algorithm 4: Profiling and equivalence detection

Input: node with big-job overhead o^{big} at depth d

Normalize loads: $\tilde{o}_i \leftarrow o_i^{\text{big}} / T_{\text{norm}}$;
Discretize $\tilde{o}_i$ using geometric bins;
Construct histogram K;
if K *already seen at depth* d **then**
    Mark node as having similar profile;
else
    Store K as seen at depth d;

Depending on the selected profiling mode, this information is used either to deprioritize the node in the priority queue or to discard it entirely before insertion.

6.3 Flexible instance generation and experimental configuration

The inherited experimental setup allowed testing multiple problem sizes by specifying pairs $(n_{\text{jobs}}, n_{\text{machines}})$ and iterating over a fixed range of random seeds. While sufficient for basic validation, this approach provided limited flexibility when exploring the behavior of the algorithm under different parameter combinations.

To support a more systematic and exploratory experimental analysis, the instance-generation pipeline was extended to decouple *instance specification* from *algorithmic configuration*. In the revised setup, an experiment is defined by explicitly pairing a concrete problem instance with a specific set of algorithmic parameters.

More precisely, each experiment is characterised by:

- the number of machines and jobs,
- the distribution and range of processing times,
- the random seed used to generate a concrete realization,
- and the algorithmic configuration under which the instance is evaluated (including profiling mode, bounding strategy, branching rule, rounding rule, node selection strategy,

and tolerance ε).

This separation enables fine-grained control over the experimental space. The same instance can be evaluated under multiple algorithmic variants, and conversely, specific configurations can be restricted to carefully selected families of instances. This flexibility proved essential during the exploratory phase of the project, allowing the identification of meaningful parameter ranges and combinations worthy of deeper analysis.

In addition, to reduce computational overhead and enable large-scale testing, exact optimal solutions were no longer computed for every instance. Instead, the fractional optimum for identical machines, given by $\sum_j p_j/m$, was used as a consistent baseline for performance evaluation. This choice is sufficient for measuring approximation quality and aligns with the theoretical framework of the PTAS, while significantly improving experimental throughput.

Finally, this setup made it possible to directly compare configurations with and without profiling under comparable approximation guarantees. Since profiling introduces a more relaxed stopping condition, non-profiling configurations were evaluated using a correspondingly adjusted tolerance, ensuring that all comparisons refer to equivalent approximation levels.

6.4 Analysis and data exploration pipeline

All experimental runs were automatically logged into structured `CSV` files, with each row corresponding to a single execution of the branch-and-bound algorithm under a specific instance–configuration pair. The recorded data include instance characteristics, algorithmic parameters, runtime statistics, search metrics (e.g. explored nodes, depth), and solution quality indicators.

To analyze and interpret these results, a set of Python notebooks was developed. These notebooks served as an exploratory and analytical layer on top of the raw experimental data, supporting data filtering, aggregation, visualization, and comparison across configurations. Standard data-analysis libraries were used to load the `CSV` files, manipulate the datasets, and generate plots illustrating trends and performance trade-offs.

The notebooks were primarily used for two purposes. First, they enabled an exploratory analysis phase, during which significant ranges of parameters (e.g. tolerance ε, instance sizes, and profiling modes) were identified. This phase guided the selection of meaningful experimental configurations and helped exclude combinations that were either uninformative or computationally impractical.

Second, once the experimental scope was consolidated, the same analysis pipeline was used to produce the plots and summary statistics presented in Chapter 7. In this phase, the notebooks acted as a reproducible tool for aggregating results and visualizing the impact of profiling, tolerance levels, and instance scale on both runtime and solution quality.

The separation between raw data collection and analysis ensured that experimental results could be re-examined, filtered, and visualized without rerunning the computationally expensive experiments, and facilitated an iterative workflow between experimentation and analysis.

7 Analysis of results

In this chapter we analyze the experimental results obtained by comparing the base branch-and-bound algorithm (`NO_PROFILING`) with its profiling-enhanced variant (`PRUNE`) for the identical machines scheduling problem. The objective is to experimentally validate the effectiveness of profiling as a mechanism to reduce the explored search space and to extend the practical applicability of the algorithm to larger and more demanding instances.

7.1 Experimental setup recap

After an extensive exploratory phase, a consolidated experimental configuration was selected in order to highlight the behavioral differences between the two variants under comparable approximation guarantees. The experiments were conducted as follows:

- For each problem size, **200 independent instances** were generated using different random seeds.
- The number of machines was progressively increased:

$$m \in \{5, 25, 50, 75, 100\},$$

  with the number of jobs fixed to $n = 2m$, as this ratio empirically represents a non-trivial and challenging regime for identical machines.
- Job processing times were drawn from a **uniform integer distribution** in $[1, 5000]$.
- A single algorithmic configuration was used for all runs, varying only the profiling mode and the approximation tolerance:
  - `NO_PROFILING` with tolerance 2ε
  - `PRUNE` with tolerance ε
- This choice ensures a fair comparison, as profiling introduces an additional approximation factor and therefore requires a relaxed stopping criterion.
- To keep runtimes within reasonable bounds and enable large-scale experimentation, the search was capped at **2000 explored nodes**.

The main performance metric considered is the **number of explored nodes**, which is a

more stable and implementation-independent indicator of algorithmic efficiency than wall-clock runtime.

This setup is consistent with the overall methodology described in the previous chapters and allows a direct, controlled comparison between the two algorithmic variants.

7.2 Results for $\varepsilon = 0.1$ (`NO_PROFILING`: $\varepsilon = 0.2$)

Figure X (first attached plot) reports the number of explored nodes for each instance, sorted in increasing order, for both algorithm variants and increasing instance sizes.

A clear qualitative difference emerges already for moderate problem sizes:

`NO_PROFILING`.

- For $m = 5$, all instances are solved well below the node limit.
- At $m = 25$, a significant fraction of instances already reaches the maximum of 2000 explored nodes.
- For $m = 50$, the vast majority of instances hit the node cap, indicating that the algorithm is no longer able to effectively prune the search space.
- For $m = 75$ and $m = 100$, more than 80–90% of instances fail to terminate within the imposed limit.

`PRUNE` **(profiling enabled).**

- For $m = 25$, almost all instances are solved with a relatively small number of explored nodes.
- At $m = 50$, only a limited tail of hard instances approaches the node cap.
- Even for $m = 75$, the majority of instances still terminate successfully.
- At $m = 100$, although the problem becomes demanding, profiling allows a non-negligible fraction of instances (roughly 40–50%) to be solved within the node limit, whereas the base version becomes ineffective.

These results clearly show that profiling delays the onset of combinatorial explosion and preserves algorithmic viability for significantly larger instances.

7.3 Stricter tolerances: $\varepsilon = 0.05$ and $\varepsilon = 0.01$

Figure Y (second attached plot) reports analogous experiments performed with tighter approximation requirements:

$$\varepsilon = 0.05/0.1 \qquad \text{and} \qquad \varepsilon = 0.01/0.02.$$

As expected, reducing ε substantially increases the computational burden for both variants. However, the relative advantage of profiling becomes even more pronounced:

- Without profiling, the algorithm exhibits poor scalability already at $m = 25$, with a sharp increase in failed instances.
- For $\varepsilon = 0.01$, the `NO_PROFILING` variant is effectively unusable beyond very small instances.
- In contrast, the profiling-enhanced version remains viable for small and medium-sized instances, even under these stricter precision requirements, and consistently explores one order of magnitude fewer nodes than the base version in comparable settings.

This behavior confirms that profiling is particularly effective in regimes where high precision exacerbates the branching complexity, precisely the scenarios where a PTAS-style pruning strategy is theoretically motivated.

7.4 Discussion and insights

Several important observations can be drawn from these results:

- **Profiling dramatically reduces the effective search space.** The reduction in explored nodes is systematic and grows with instance size, confirming that profile-based equivalence detection successfully eliminates large sets of redundant subproblems.
- **The benefit increases with problem difficulty.** For easy instances, both variants perform similarly. The advantage of profiling becomes substantial exactly when the base algorithm begins to struggle.
- **Runtime trade-off is secondary.** While profiling introduces a per-node overhead due to profile computation and comparison, this cost is largely dominated by the savings obtained through reduced exploration. When the number of explored nodes is similar, profiling is slightly slower; however, in all practically relevant hard cases, the base algorithm fails outright, while the profiling version remains tractable.
- **Profiling effectively extends the usable range of the algorithm.** Under the imposed node limit, profiling shifts the boundary of solvable instances by roughly one to two problem sizes, depending on ε. This extension is crucial in practice and aligns with the theoretical role of profiling within the PTAS framework.

7.5 Summary

The experimental results provide strong empirical evidence that profiling is a highly effective enhancement for branch-and-bound applied to identical machines scheduling. By aggressively pruning structurally similar subproblems, profiling significantly reduces the explored search space and allows the algorithm to scale to larger instances and tighter approximation

guarantees. These findings experimentally validate the theoretical motivations behind profiling and confirm its practical relevance within a PTAS-oriented branch-and-bound framework.

Overall, profiling increases the maximum practically solvable instance size by approximately one to two problem scales under a fixed exploration budget, turning otherwise intractable configurations into feasible ones.

Indicative failure rates at the node cap

Table 7.1: Approximate percentage of instances reaching the 2000-node limit (200 instances per size).

m	n	NO_PROFILING	PRUNE
5	10	0%	0%
25	50	60–80%	0–5%
50	100	90–100%	10–30%
75	150	90–100%	20–40%
100	200	90–100%	40–50%

8 Methodology

This chapter describes the collaborative approach adopted throughout the project. Given the modest scope of the work, both team members were involved in all aspects, with small task divisions made primarily to improve efficiency and ensure comprehensive coverage.

8.1 Division of responsibilities

The project was approached collaboratively, with both team members working together on all major components. While we jointly addressed the theoretical foundation, algorithm design, and implementation, we occasionally divided smaller tasks to work in parallel and increase efficiency.

For example, when implementing the problem variant (identical job scheduling), we would sometimes work on different parametrizations simultaneously, then review and discuss each other's work to ensure consistency and share insights.

This approach provided natural redundancy and allowed us to catch potential issues early through continuous peer review.

8.2 Planning and task management

The project was organized into basic phases: initial research and problem study, algorithm design, implementation, experimental evaluation, and documentation. We maintained a simple timeline with key milestones to track progress.

Task management was kept lightweight, using Git for version control and informal tracking of what needed to be done. We prioritized understanding the theoretical framework before implementation to ensure our algorithms were correctly designed.

The development followed an iterative approach, refining implementations based on experimental results and discussions about observed performance.

8.3 Pair programming and brainstorming

Most of the work was done collaboratively through frequent discussions and pair programming sessions. This was particularly helpful when tackling complex aspects like designing the profiling algorithm.

We regularly discussed trade-offs between solution quality and computational efficiency, especially when balancing greedy heuristics with exact methods. Having two perspectives helped us identify issues and find better solutions.

8.4 Weekly progress meetings

Weekly meetings with our relators were held throughout the project to discuss progress, present findings, and receive guidance on both theoretical and practical aspects. These sessions were helpful for ensuring the work remained on track and aligned with academic standards.

During these meetings, we presented experimental results, discussed implementation challenges, and received valuable feedback that helped shape our approach. The relators' expertise was particularly helpful in validating our algorithmic choices and suggesting areas for deeper investigation.

These regular checkpoints also served as opportunities to clarify theoretical concepts, validate that our implementation was correct, and ensure our experimental methodology was rigorous and appropriate for the research objectives.

8.5 Data collection and analysis

Experimental data was collected across multiple problem instances with varying characteristics (problem size, machine counts, value distributions). We designed experiments to validate theoretical complexity bounds and measure practical performance.

Instance generation was automated using Python scripts with controlled random seeds to ensure reproducibility. Results were logged in CSV format for easy analysis.

We used Jupyter notebooks for data visualization and analysis, creating plots to understand algorithm behavior and performance trends. This helped us verify that the implementation met the theoretical guarantees and performed as expected.

9 Conclusions

This project provided the opportunity to work on a topic that is both conceptually challenging and uncommon in typical coursework. Implementing and experimentally validating advanced algorithmic ideas required sustained effort, careful design choices, and continuous iteration between theory and practice.

Overall, the results obtained were highly satisfactory. The experimental analysis confirms that the profiling-based optimizations have a tangible impact on the behavior of the branch-and-bound algorithm, reducing redundant exploration and enabling high-quality approximate solutions to be obtained within practical computational limits. These findings align well with the theoretical expectations and reinforce the practical relevance of the proposed approach.

One of the main lessons learned concerns the gap between theoretical descriptions and concrete implementations. Concepts that are compactly expressed in mathematical form often require non-trivial interpretation when translated into code, and their effectiveness can depend sensitively on implementation details. In this context, empirical evaluation proved essential for understanding which design choices truly matter in practice.

Focusing on the identical machines setting turned out to be a sound and effective choice. It allowed the impact of profiling-based strategies to be studied in isolation, without introducing unnecessary complexity, and led to clearer and more interpretable results.

With hindsight, additional effort could have been devoted to a deeper characterization of instance hardness. Rather than expanding the set of algorithmic variants, a broader and more systematic exploration of instance distributions and structural properties could help identify which classes of instances are genuinely challenging for the algorithm, and under which conditions the proposed optimizations are most beneficial.

In conclusion, this project was both demanding and rewarding, offering valuable insights into the practical behavior of theoretically motivated algorithms and highlighting the importance of careful experimental design when evaluating approximation-oriented methods.

Bibliography

[1] Koppányi István Encz, Monaldo Mastrolilli, and Eleonora Vercesi. Branch-and-bound algorithms as polynomial-time approximation scheme. *Mathematics of Operations Research*, 2025.

[2] Scip optimization suite. `https://scipopt.org`.